

# Review Scoring API — Functional Specification

Content classification and reviewer enforcement triage · v1.0

| Field      | Value                           | Field          | Value                                       |
|------------|---------------------------------|----------------|---------------------------------------------|
| Author     | Jadon Yeo                       | Status         | Draft for review                            |
| Version    | 1.0                             | Date           | 22 September 2026                           |
| Applies to | Review Scoring API v1           | Interface spec | review-scoring-api.v1.yaml                  |
| Reviewers  | Engineering, Trust & Safety Ops | Related        | github.com/bobags2/yelp-review-intelligence |

This document accompanies the OpenAPI specification for the Review Scoring API. The OpenAPI file defines the interface contract; this document defines what the service is for, who consumes it, what it must do to be considered correct, and which decisions still need a business owner.

**It is written as a work sample.** Section 8 records honestly which parts of the specification are implemented in the reference service and which are specified but not yet built, because a specification that quietly overstates the implementation is the failure mode this document exists to prevent.

## 1. Purpose and scope

The Review Scoring API serves two consumers with one transport and one error contract. It exists so that teams reviewing user-generated content can prioritise their attention, and so that content can be classified without each consuming team building and maintaining its own model.

### 1.1 In scope

- Multi-label classification of review text against a published business-category vocabulary.
- Behavioural scoring of a reviewer account from caller-supplied aggregates, returning a queue score and its per-signal contributions.
- Operational endpoints for liveness and rolling latency telemetry.
- The error contract, batch limits and reproducibility guarantees that make both usable by an integrating client without a support call.

### 1.2 Out of scope

- Any automated enforcement action. The service returns scores; every decision to act is made by a person or by a separate policy service.
- Threshold selection. The service applies none — see OQ-01.
- Computation of reviewer behavioural aggregates. The caller computes them where the review history already lives; the scoring service stays stateless.
- Storage of review text. Request bodies are not persisted — see OQ-05 for the logging question this leaves open.

### 1.3 A scope decision worth recording

The obvious framing of this product is a supervised fraudulent-review detector. That framing was ruled out during discovery: the available corpus carries no ground-truth labels for fraudulent reviews, so a supervised model could only have been trained against heuristic labels invented for the purpose — a model that learns to reproduce its own rules and reports high accuracy for doing so.

The scope was therefore split. Classification runs supervised against labels that genuinely exist. Reviewer scoring runs unsupervised and produces queue candidates for human adjudication, which is how enforcement systems are actually operated: a cheap signal orders the queue, people work the top of it, and their decisions become the labels a supervised model can later be trained on.

## 2. Stakeholders and consumers

| Stakeholder                  | Interest                                     | What they need from this service                             |
|------------------------------|----------------------------------------------|--------------------------------------------------------------|
| Trust & Safety operations    | Works the enforcement queue                  | Ranking by impact, and a visible reason for every flag       |
| Content / catalogue team     | Category quality on listings                 | Batch classification with calibrated per-label probabilities |
| Integrating client engineers | Builds against the API                       | A stable contract, machine-readable errors, correlation ids  |
| Compliance / audit           | Reviews enforcement decisions after the fact | Reproducible scores tied to a recorded model version         |
| Platform operations          | Runs the service                             | Honest health signalling and stage-level latency telemetry   |

## 3. End-to-end client journey

The integration path a new consuming team follows, and the point at which each currently stalls. Steps 2 and 5 are where onboarding support is most often needed.

| Step | Stage             | What has to be true for the client to get past it                                                                                                                                         |
|------|-------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 1    | Discovery         | The client can tell from the specification alone whether the service answers their question. Requires the "scores are not decisions" framing to be stated up front, not discovered later. |
| 2    | Credentials       | An environment-scoped key is issued, and the sandbox returns the same contract as production so integration work is not redone. See OQ-04.                                                |
| 3    | First call        | A worked example copies and runs. Every operation in the specification carries one.                                                                                                       |
| 4    | Batch integration | The client knows the batch limit, and knows which batch size to choose — the specification carries throughput guidance because the answer is not intuitive (see NFR-03).                  |
| 5    | Error handling    | The client can branch on a stable error identifier rather than parse a prose message. Requires the RFC 9457 envelope.                                                                     |
| 6    | Production        | Rate limits are published, degradation is signalled honestly, and correlation ids let a client-side failure be traced to a server log.                                                    |

## 4. User stories and acceptance criteria

*Acceptance criteria are written to be executable as tests. Each is a statement that can be shown to pass or fail without interpretation.*

### Epic A — Content classification

**US-01 As a catalogue analyst, I want a batch of reviews classified by business category so that I can find reviews posted against the wrong type of business.**

**Given** a batch of 1 to 64 reviews, when submitted, then results are returned positionally aligned with the request array.

**Given** a review scored successfully, when returned, then predictions are ordered by descending probability and contain at most topK entries.

**Given** a review about a business belonging to several categories, when scored, then probabilities are independent per label and are not required to sum to 1.

**US-02 As an integrating client engineer, I want one unusable review not to fail the whole batch so that a single bad record does not cost me 63 good results.**

**Given** a batch containing one review too short to score, when submitted, then the response status is 200.

**And** the unusable item occupies its position with an empty predictions array and a populated skippedReason.

**And** every other item in the batch returns predictions normally.

**US-03 As a capacity planner, I want to know which batch size to send so that I am not guessing at a value that materially changes my throughput.**

**Given** the published specification, when a client reads the score operation, then throughput guidance states the measured optimum and the reason it is not the maximum.

**Given** a response, when returned, then latency is decomposed into tokenisation and inference so the client can tell which stage dominates.

## **Epic B — Reviewer enforcement triage**

**US-04 As an enforcement queue owner, I want accounts ranked by expected impact rather than by how unusual they are so that reviewer time goes to the accounts that cost the most if nobody looks.**

**Given** two accounts with equal anomalyScore and different reviewCount, when both are scored, then the higher-volume account returns the higher queueScore.

**Given** an account with 3 reviews at a duplication ratio of 0.67, when scored, then its duplication contribution is shrunk toward the population mean relative to an account with 200 reviews at the same ratio.

**Given** any successful response, when returned, then both anomalyScore and queueScore are present and separately addressable.

**US-05 As a moderator, I want the per-signal breakdown returned with the score so that I can see why an account was flagged before I take action against it.**

**Given** any successful reviewer score, when returned, then components contains one entry per submitted signal.

**And** the component values sum to anomalyScore within floating-point tolerance.

**US-06 As a compliance reviewer, I want a score to be reproducible so that an enforcement decision taken today can be explained in an audit twelve months from now.**

**Given** identical inputs and the same modelVersion, when scored on two separate occasions, then the returned anomalyScore is identical.

**Given** any successful response, when returned, then modelVersion is populated and identifies the scaling table used.

**Given** a new model release, when deployed, then previously issued scores remain explainable because the superseded scaling table is retained.

**US-07 As an integrating client engineer, I want a partial set of signals rejected rather than defaulted so that I find out about my bug instead of receiving a plausible wrong number.**

**Given** a request omitting one of the five required signals, when submitted, then the response is 422 and no score is returned.

**And** errors identifies the missing field by JSON Pointer.

## **Epic C — Integration and operations**

**US-08 As an integrating client engineer, I want machine-readable errors so that I can handle failures in code without parsing prose.**

**Given** any 4xx or 5xx response, when returned, then the content type is application/problem+json and a stable type URI is present.

**Given** a validation failure, when returned, then errors contains one entry per offending field with a JSON Pointer.

**Given** a batch exceeding the documented maximum, when submitted, then the response is 413 and states the maximum, so the client can split rather than retry unchanged.

**US-09 As an integrating client engineer, I want a correlation id echoed so that I can trace a failure on my side to a log line on yours.**

**Given** a request carrying X-Request-Id, when it completes or fails, then the same value is returned in the response header.

**Given** a request without one, when it completes, then a generated id is returned and recorded server-side.

**US-10 As a platform operator, I want health to distinguish "running" from "able to serve" so that an instance which has not loaded its model is drained rather than sent traffic.**

**Given** a capability whose model has not loaded, when health is called, then status is degraded and that capability is reported unavailable with a reason.

**Given** a request to an unavailable capability, when submitted, then the response is 503 with Retry-After, and is safe to retry.

## 5. Non-functional requirements

Every figure below is traceable to a recorded run on the reference deployment rather than to an estimate. Where a number is arithmetic derived from measurements rather than directly observed, the basis column says so.

| Ref    | Requirement          | Target                                                                            | Basis                                                                                                                                                                                                       |
|--------|----------------------|-----------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| NFR-01 | Sustained throughput | $\geq 11$ reviews/s per instance at batch 8, CPU execution provider               | Measured: 2,006 events in 175.5 s on the reference deployment                                                                                                                                               |
| NFR-02 | Batch latency        | p95 of a batch-8 call $\leq 400$ ms on production-representative text             | Measured model cost at batch 8, plus margin. Real review text costs $\approx 4.5\times$ the synthetic benchmark corpus per review because it reaches the token cap; the benchmark understates absolute cost |
| NFR-03 | Scaling constraint   | Per-batch pipeline overhead $\approx 309$ ms, which exceeds model cost at batch 8 | Measured. Consequence: a throughput target materially above NFR-01 requires reducing per-batch overhead, not a faster model. Recorded so that a future capacity request is not answered with the wrong work |
| NFR-04 | Reproducibility      | Identical inputs and model version return an identical score                      | Verified to $1e-06$ across 500 sampled accounts, batch path against serving path                                                                                                                            |
| NFR-05 | Batch limit          | 64 reviews per call, enforced                                                     | Specified; rejection is 413 with the maximum stated                                                                                                                                                         |
| NFR-06 | Degradation          | Unloaded capability reports unavailable rather than failing on first use          | Specified; health returns degraded with the capability named                                                                                                                                                |
| NFR-07 | Observability        | p50, p95 and p99 over a rolling window, split by processing stage                 | Implemented. Mean is reported but is not an alerting signal                                                                                                                                                 |

**A note on NFR-02 and NFR-03.** These two were originally specified from a model-only benchmark, which measures the scoring call in isolation. That benchmark is correct and was still the wrong basis: the pipeline also pays a fixed per-batch cost the benchmark cannot observe, and the two costs move in opposite directions as batch size changes. Specifying the batch size from the model curve alone halved measured throughput. The requirement was re-derived from an end-to-end run. Recorded here because the same mistake is available to anyone reading the benchmark without its scope.

## 6. Error handling and edge cases

| Status | Condition                                                                                        | Expected client behaviour                                    |
|--------|--------------------------------------------------------------------------------------------------|--------------------------------------------------------------|
| 400    | Unparseable body or unknown field                                                                | Fix and resubmit. Not retryable unchanged                    |
| 401    | Missing, malformed, or wrong-environment key                                                     | Check the key matches the environment being called           |
| 413    | Batch above the documented maximum                                                               | Split the batch. Retrying unchanged will fail identically    |
| 422    | Well-formed but semantically invalid — empty batch, signal out of range, missing required signal | Correct the fields named in errors[].pointer                 |
| 429    | Rate limit exceeded                                                                              | Back off for Retry-After. Retrying sooner extends the window |
| 503    | Capability has not loaded                                                                        | Retry with backoff. This is a startup state, not a rejection |

### 6.1 Edge cases decided rather than left implicit

- Review text longer than the model context window is truncated and scored, not rejected. Rejecting a long review would fail a valid request; the alternative was judged worse than a slightly degraded score.
- A review too short to carry signal is skipped per item, not rejected batch-wide. See US-02.
- Non-English text is accepted and scored, but the current model is trained on English only and confidence is not calibrated for it. This is a known compromise and is recorded as OQ-06 rather than left for a client to discover.

- A reviewer signal outside its documented range is rejected rather than clamped. Clamping produces a plausible score from bad input, which is harder to detect than an error.

## 7. Open questions register

Decisions that need a business owner rather than an engineering answer. Each blocks or qualifies part of the specification above.

| Ref   | Question                                                                          | Why it is not an engineering call                                                                         | Owner needed                  |
|-------|-----------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------|-------------------------------|
| OQ-01 | Who sets the probability threshold at which a score triggers a downstream action? | The cost of a false positive differs by action — a queue entry and a suspension are not the same decision | Trust & Safety policy         |
| OQ-02 | Can a client pin a model version, or does it always receive the latest?           | Pinning strengthens the audit guarantee in US-06 but slows rollout of model improvements                  | Product                       |
| OQ-03 | What per-client rate limit applies, and is it shared or per-environment?          | Depends on commercial tiering, not on capacity                                                            | Product / commercial          |
| OQ-04 | Is an API key sufficient, or is OAuth2 client-credentials required?               | Determined by the client organisation's security standard, not by the service                             | Security                      |
| OQ-05 | Review text is not stored — but is it written to logs on error?                   | Review text may contain personal data. Retention and redaction are a privacy decision                     | Privacy / compliance          |
| OQ-06 | Reject non-English text, or score it with an uncalibrated-confidence caveat?      | Trade-off between coverage and a score a consumer may over-trust                                          | Product                       |
| OQ-07 | Should the service expose a helper that computes reviewer aggregates?             | Moves work across a team boundary and changes the statelessness decision in §1.2                          | Engineering / product jointly |

## 8. Implementation status

The reference service at the linked repository implements part of this specification. The remainder is specified and not built. Recorded explicitly so that no reader mistakes the specification for a description of what currently runs.

| Item                                           | Status         | Note                                                                 |
|------------------------------------------------|----------------|----------------------------------------------------------------------|
| Four endpoints, batch scoring, per-item skip   | Implemented    | Paths differ — see below                                             |
| Per-signal component breakdown                 | Implemented    | US-05 satisfied                                                      |
| Reproducibility from a persisted scaling table | Implemented    | Verified to 1e-06, NFR-04                                            |
| Health with capability-level status            | Implemented    | US-10 satisfied                                                      |
| Rolling p50/p95/p99, split by stage            | Implemented    | NFR-07 satisfied                                                     |
| RFC 9457 problem+json envelope                 | Specified only | Service returns the framework default error shape; US-08 not yet met |
| API key authentication                         | Specified only | Blocked on OQ-04                                                     |
| Rate limiting and RateLimit headers            | Specified only | Blocked on OQ-03                                                     |
| /v1 prefix and resource:verb paths             | Specified only | Current paths are /score, /anomaly, /healthz, /metrics               |
| modelVersion in responses                      | Specified only | Required before OQ-02 can be answered                                |

## 9. Traceability

| Story        | Interface element      | Verified by                    | NFR link |
|--------------|------------------------|--------------------------------|----------|
| US-01, US-02 | POST /v1/reviews:score | Positional alignment and skip- | NFR-05   |

| Story               | Interface element                     | Verified by                           | NFR link               |
|---------------------|---------------------------------------|---------------------------------------|------------------------|
|                     |                                       | reason tests                          |                        |
| <b>US-03</b>        | x-throughput-guidance, latency object | End-to-end throughput run             | NFR-01, NFR-02, NFR-03 |
| <b>US-04, US-05</b> | POST /v1/reviewers:score              | Ordering and component-sum assertions | —                      |
| <b>US-06</b>        | modelVersion field                    | Batch-versus-serving parity check     | NFR-04                 |
| <b>US-07, US-08</b> | Problem schema, 422 with pointers     | Validation tests per field            | —                      |
| <b>US-09</b>        | X-Request-Id parameter and header     | Echo test on success and failure      | —                      |
| <b>US-10</b>        | GET /v1/health                        | Model-absent startup test             | NFR-06                 |

*Prepared by Jadon Yeo as a work sample. The reference implementation, its measurements, and the written analysis behind the figures in section 5 are public at [github.com/bobags2/yelp-review-intelligence](https://github.com/bobags2/yelp-review-intelligence).*